



2027 Nurture Partner Network (NPN) MuleSoft

Hackathon Challenge – IoT Sensor Data Ingestion

Hackathon Guidelines

1. Each team shall complete and submit the hackathon project within five calendar days, commencing on the date the hackathon requirements are communicated.
2. All developed APIs and apps must be in a runnable state, with MUnit tests included.
3. Every team member must have the runnable code on their own laptop.
4. Each member can have their own way of implementation as well.
5. Running the API on your Anypoint trial account runtime is highly preferred. If you face any issue with your Anypoint account, run the app on a local setup instead — note that a local run will be scored lower than a CloudHub runtime deployment.

Technical Standards and Best Practices

1. Prepare High Level Design document.
2. API-LED Architecture.
3. Global error handling.
4. Use RAML wherever applicable and API Kit router.
5. MUnit covering 100% of components. If not possible, need valid reasons.
6. Auto Discovery.
7. Externalizing properties, secure properties and dwl files.
8. Appropriate Logging levels.
9. Apply applicable API policies for performance & security.
10. Use of appropriate private and sub flows.
11. Use core components based on the need like dwl, for-each/parallel for-each, etc.,
12. Domain app for studio run applications.
13. Basic alerts, logging & monitoring.
14. Code should be maintained in Git. And everyone should be able to commit, pull, raise merge requests, merge conflicts, etc.,
15. Use batch processing, caching, object store wherever required in your project.
16. Standard names for components in the flow.

Scenario: IoT - Sensor Data Ingestion

Sensor payloads are received over HTTP, buffered in a queue to handle burst traffic, and upserted into a time-series database for real-time analytics dashboards.

The ingestion endpoint must survive very uneven traffic: a fleet of devices may reconnect at once after a network outage and replay buffered readings. The queue therefore acts as a shock absorber, and the consumer must write to the database in batches rather than one row per reading. Readings can arrive out of order and can be duplicated by device-side

retries, so the database write must be an upsert keyed on device, metric and reading timestamp.

Components to Be Used

1. HTTP Listener/Requestor
2. Queue
3. Database

Functional Requirements

1. **Telemetry Ingestion API:** Expose a POST /telemetry endpoint through an HTTP listener, defined in RAML and routed with the APIKit router. The endpoint must accept both a single reading and a batch of readings from one device.
2. **Payload Validation:** Validate the device ID, metric name, numeric value, unit and reading timestamp. Reject readings whose timestamp is more than a configurable number of minutes in the future or older than a configurable retention window.
3. **Range Validation:** Validate each metric value against configured minimum and maximum bounds per metric type, and mark out-of-range readings with a quality flag rather than discarding them silently.
4. **Device Authentication:** Require an API key or client credentials on the ingestion endpoint and apply the corresponding API policy. Reject readings from an unknown or deactivated device.
5. **Queue Buffering:** Publish accepted readings to a telemetry queue immediately and return HTTP 202 Accepted. The API must not write to the database directly.
6. **Batch Consumer:** Consume from the queue and accumulate readings into batches using batch processing or an aggregator, with a configurable batch size and timeout, so that the database receives bulk inserts instead of single row writes.
7. **Upsert Logic:** Write each reading with an upsert keyed on device ID, metric name and reading timestamp, so that a replayed reading updates the existing row instead of creating a duplicate.
8. **Latest Value Table:** Maintain a separate current readings table holding only the most recent value per device and metric, for dashboard queries. Only update it when the incoming reading is newer than the stored one.
9. **Threshold Evaluation:** Compare each reading against configured warning and critical thresholds and record a breach status on the row, so that the dashboard can highlight abnormal devices.
10. **Back Pressure Handling:** Cap the in-flight message count and use a configurable consumer concurrency so that a burst does not exhaust the database connection pool.
11. **Error Handling:** Apply a global error handler. Route readings that fail validation or persistence after retries to a dead-letter queue for later inspection, and continue processing the rest of the batch.

12. **Logging and Monitoring:** Log batch size, processing duration and rejected record counts. Configure alerts on queue depth and on a sustained drop in ingestion rate, which usually indicates device connectivity loss.
13. **Unit Testing:** Provide MUnit tests for validation, batching, upsert behaviour with duplicate and out-of-order readings, and threshold evaluation, with the queue and database mocked.

Sample Telemetry Request

```
{
  "deviceId": "DEV-CHN-00417",
  "siteId": "SITE-CHN-02",
  "firmwareVersion": "3.2.1",
  "batchId": "BATCH-2026-09-11-0091",
  "readings": [
    {
      "metricName": "TEMPERATURE",
      "value": 78.4,
      "unit": "CELSIUS",
      "readingTimestamp": "2026-09-11T09:15:00+05:30",
      "quality": "GOOD"
    },
    {
      "metricName": "VIBRATION",
      "value": 4.12,
      "unit": "MM_PER_SEC",
      "readingTimestamp": "2026-09-11T09:15:00+05:30",
      "quality": "GOOD"
    },
    {
      "metricName": "PRESSURE",
      "value": 2.05,
      "unit": "BAR",
      "readingTimestamp": "2026-09-11T09:15:30+05:30",
      "quality": "GOOD"
    }
  ]
}
```

Expected API Response

```
HTTP/1.1 202 Accepted
Content-Type: application/json
{
  "batchId": "BATCH-2026-09-11-0091",
  "deviceId": "DEV-CHN-00417",
  "acceptedCount": 3,
  "rejectedCount": 0,
  "rejected": [],
  "status": "QUEUED",

  "correlationId": "e1c48a90-7bd2-4f55-a6d3-9418ffb20c77"
}
```

Database Table Structure

```
CREATE TABLE sensor_readings (  
  device_id    VARCHAR(30) NOT NULL,  
  metric_name  VARCHAR(40) NOT NULL,  
  reading_timestamp TIMESTAMP NOT NULL,  
  metric_value DECIMAL(16,4) NOT NULL,  
  unit        VARCHAR(20) NOT NULL,  
  quality_flag VARCHAR(15) NOT NULL,  
  breach_status VARCHAR(15),  
  site_id     VARCHAR(30),  
  firmware_version VARCHAR(20),  
  ingested_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  correlation_id VARCHAR(50),  
  PRIMARY KEY (device_id, metric_name, reading_timestamp)  
);  
  
CREATE TABLE sensor_current_value (  
  device_id    VARCHAR(30) NOT NULL,  
  metric_name  VARCHAR(40) NOT NULL,  
  metric_value DECIMAL(16,4) NOT NULL,  
  unit        VARCHAR(20) NOT NULL,  
  reading_timestamp TIMESTAMP NOT NULL,  
  breach_status VARCHAR(15),  
  updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (device_id, metric_name)  
);  
  
CREATE TABLE device_registry (  
  device_id    VARCHAR(30) PRIMARY KEY,  
  site_id     VARCHAR(30) NOT NULL,  
  device_status VARCHAR(15) NOT NULL,  
  api_key_hash VARCHAR(128),  
  registered_at TIMESTAMP  
);
```

Queue Message Format

```
{  
  "correlationId": "e1c48a90-7bd2-4f55-a6d3-9418ffb20c77",  
  "eventType": "TELEMETRY_BATCH",  
  "deviceId": "DEV-CHN-00417",  
  "siteId": "SITE-CHN-02",  
  "batchId": "BATCH-2026-09-11-0091",  
  "receivedAt": "2026-09-11T09:15:02.117+05:30",  
  "readingCount": 3,  
  "readings": [  
  
    { "metricName": "TEMPERATURE", "value": 78.4, "unit": "CELSIUS",  
      "readingTimestamp": "2026-09-11T09:15:00+05:30",  
      "quality": "GOOD" },  
    { "metricName": "VIBRATION", "value": 4.12, "unit": "MM_PER_SEC",  
      "readingTimestamp": "2026-09-11T09:15:00+05:30",  
      "quality": "GOOD" },  
    { "metricName": "PRESSURE", "value": 2.05, "unit": "BAR",  
      "readingTimestamp": "2026-09-11T09:15:30+05:30",  
      "quality": "GOOD" }  
  ]  
}
```

```
]
}
```

Threshold Configuration Example

```
# externalized in metric-thresholds.yaml
metrics:
  TEMPERATURE:
    unit: CELSIUS
    min: -20.0
    max: 120.0
    warning: 75.0
    critical: 95.0
  VIBRATION:
    unit: MM_PER_SEC
    min: 0.0
    max: 50.0
    warning: 4.0
    critical: 8.0
  PRESSURE:
    unit: BAR
    min: 0.0
    max: 10.0
    warning: 6.0
    critical: 8.5

ingestion:
  batchSize: 500
  batchTimeoutMillis: 2000
  consumerConcurrency: 4
  maxFutureSkewMinutes: 5
  retentionWindowDays: 30
```

Basic Negative Scenarios to Be Covered

1. **Invalid or incomplete telemetry payload:** Reject malformed JSON, a missing device ID, metric name, value or timestamp, and non-numeric values. Return a client error identifying the rejected readings.
2. **Unknown or deactivated device:** Reject readings from a device that is absent from the registry or marked inactive, and log the attempt without creating any database rows.
3. **Out-of-range metric value:** Flag a value outside the configured bounds with a quality indicator, and confirm that it does not overwrite a good current value on the dashboard table.
4. **Future-dated or expired timestamp:** Reject readings dated beyond the allowed clock skew or older than the retention window, and report them in the rejected list of the response.
5. **Duplicate reading replay:** Send the same device, metric and timestamp twice and confirm that the upsert results in exactly one row and no duplicate dashboard update.
6. **Out-of-order readings:** Send an older reading after a newer one and confirm that the current value table retains the newer value.

7. **Burst traffic and queue backlog:** Push a large burst of readings and confirm that the API still responds within target, the queue absorbs the backlog, and no message is lost or the connection pool exhausted.
8. **Queue unavailable or publish failure:** Retry the publish within limits and return an appropriate error rather than silently dropping readings.
9. **Database connectivity or timeout failure:** Retry transient failures and leave the queue message unacknowledged so that the batch is redelivered instead of lost.
10. **Partial batch failure:** When one reading in a batch fails to persist, continue with the remaining readings and route only the failing record to the dead-letter queue.
11. **Unexpected processing error:** Use a global error-handling strategy to log a consistent error with the correlation ID, device ID and batch ID.
12. **Additional negative scenarios:** Teams are encouraged to identify, document, and implement any additional negative scenarios relevant to their solution.

Acceptance Criteria

1. A valid single reading and a valid batch are both accepted and return HTTP 202 with accepted and rejected counts.
2. Readings appear in the readings table with the correct quality and breach status derived from the threshold configuration.
3. The current value table holds exactly one row per device and metric, always reflecting the newest reading.
4. Replaying a batch produces no duplicate rows, demonstrating a working upsert.
5. A sustained burst is absorbed by the queue, and the database receives bulk writes rather than single row inserts.
6. Invalid readings are reported back to the caller and the valid readings in the same request are still processed.
7. With the database stopped, no messages are lost and processing resumes automatically once it is available.
8. Thresholds and batch settings can be changed in the externalized configuration without any code change.
9. MUnit tests cover validation, batching, duplicate replay and out-of-order handling with mocked dependencies.